Agentic AI Stack Dictionary

The Comprehensive Reference

650+

Tools & standards
mapped and compared

36

Layers
of the agentic stack

01 Orchestration

02 MCP Protocols

03 Inference & KV Cache

04 Vector & RAG

05 Sandboxes

06 Evals & Guardrails

and 30 more categories

Contents

39 parts across the agentic engineering stack

01	How to Use This Dictionary	1
02	Agent Frameworks and Orchestration	3
03	Coding Agents and Development Environments	9
04	Foundation Models and Model Providers	14
05	Model Serving, Inference, and Optimization	18
06	Agent Protocols, Tool Connectivity, and Interoperability	22
07	RAG, Retrieval, Search, and Knowledge Systems	27
08	Vector Databases and Graph Databases	31
09	Memory, State, and Context Engineering	34
10	Browser Automation and Computer-Use Agents	37
11	Workflow Engines and Durable Execution	40
12	Low-Code, No-Code, and Visual Agent Builders	43
13	Evaluation, Benchmarking, and Testing	46
14	Observability, Tracing, and Monitoring	51
15	Security, Guardrails, and Red Teaming	54
16	Multimodal AI: Vision, Audio, and Video	57
17	Data Engineering, Document Processing, and Synthetic Data	60
18	Cloud, Containers, Deployment, and Infrastructure	63
19	API Gateways, Identity, and Application Backends	66
20	Fine-Tuning, Training, and Model Development	70
21	Reinforcement Learning and Agent Training Environments	73
22	Agent Skills, Reusable Capabilities, and Conventions	75
23	Multi-Agent Coordination and Distributed Agent Systems	78
24	Human-in-the-Loop Systems and Approval Workflows	81
25	Voice Agents and Realtime Interaction	83
26	AI Agent Marketplaces and Integration Ecosystems	86
27	Research, Search, and Web Intelligence	89
28	Database and Data-Access Tools for Agents	92
29	Testing, Software Quality, and Agent Reliability	95
30	Cost Optimization, Caching, and Performance	98
31	AI Application and Agent User Interfaces	100
32	Sandboxes and Code Execution Runtimes	102
33	Structured Output and Prompt Programming	105
34	Agent Identity, Authorization, and Governance	108
35	Agent Commerce, Payments, and Web Standards	111
36	Managed Agent Platforms and Enterprise Runtimes	113
37	Emerging Technologies and Future Trends	116
38	Reference Architectures and Decision Guides	118
39	Appendix: Quick Reference, Decision Matrices, and Glossary	124
A–Z	Index	128

01

PART I

How to Use This Dictionary

architecture, taxonomy, reading guides

An AI agent is a software system, not a model. It pairs a language model with state, tools, memory, sandboxes, guardrails and evaluation, and each of those layers has its own fast-moving market of frameworks, runtimes and services. This dictionary maps that market: 658 entries covering more than 800 named tools, standards and services across 36 layers, each with a type, licence, primary language, maintenance status and a one-line recommendation.

How it was built. Entries started from an audited master catalogue that removed duplicates and assigned every tool a single primary home. Fast-moving facts (model lineups, renames, acquisitions, archived projects, protocol governance) were re-checked against public sources on **25 October 2026**. Anything that was changing at that date is marked *Watch*. Licences are given as the project's main licence; dual-licensed and open-core products show both. Always confirm the licence in the repository before you ship.



Reading an Entry

Every chapter ends with a catalogue table:

- **Tool and type.** The name, with a type label: Framework, SDK, Library, Runtime, Protocol, Spec, Model, Platform, Service, Infra, DB, Benchmark or Tool.
- **What it does and Best for.** One factual sentence, then the workload where it is the strongest choice.

- **Licence.** Apache-2.0, MIT and BSD are permissive. GPL and AGPL are copyleft. BSL, FSL, SSPL and ELv2 are source-available with commercial limits. Proprietary means a closed product or API.
- **Lang.** The main implementation or SDK language. *API* means a hosted service; *Weights* means downloadable model weights.

Status.

• Active: actively developed and safe to adopt.

◦ Watch: renamed, recently acquired, transitioning or unstable. Re-check before you commit.

⟋ Legacy: maintenance mode, superseded or archived. Plan a migration.

Reading Guide by Role

- **AI application engineers.** Parts 02 (Frameworks), 06 (Protocols), 07 (RAG), 09 (Memory), 33 (Structured Output) and 13 (Evaluation).
- **Platform and infrastructure.** Parts 05 (Serving), 11 (Durable Execution), 18 (Cloud), 19 (Gateways), 30 (Cost) and 32 (Sandboxes).
- **Security and governance.** Parts 15 (Security), 24 (Human-in-the-Loop), 34 (Identity and Governance), 14 (Observability) and 13 (Evaluation).
- **Full-stack and product.** Parts 03 (Coding Agents), 10 (Browser Agents), 25 (Voice), 31 (UIs) and 36 (Managed Platforms).
- **Technical leaders.** Start with the reference architectures and decision flowcharts in Part 38, then the *Start Here* box at the top of each catalogue.

The Eight Capabilities

Every production agent has to cover eight things: **perception** (turning pages, audio and API responses into context), **reasoning**, **planning**, **tool execution**, **memory**, **guardrails**, **sandboxing** and **evaluation**. The chapters follow the stack from models upward, so you can check each capability against the layer that provides it.

02

PART II

Agent Frameworks and Orchestration

the execution foundation of autonomous systems

Agent frameworks provide the programming abstractions, state-management engines, and control-flow primitives required to transform stateless language models into durable, multi-step autonomous systems. Rather than executing isolated prompt-response pairs, modern agent frameworks orchestrate cyclical execution graphs: continuously observing environments, maintaining conversational state, dynamically selecting and invoking tools, evaluating outcomes, and self-correcting upon failure.

In 2026, the framework ecosystem has transitioned from early sequential chains towards directed graphs and cyclical state machines with reliable checkpointing and human-in-the-loop interruption capabilities. Frameworks prioritize type safety, streaming telemetry, dependency injection, and formal tool connectivity standards like the Model Context Protocol (MCP).

Core Architectural Archetypes

1. **Graph-Based State Machines (LangGraph, Dapr Agents).** Model the agent lifecycle as a cyclical directed graph where nodes represent computational steps (LLM calls, tool executions, data mutations) and edges represent conditional routing logic. State is formally defined using typed schemas and persisted at each turn, enabling time-travel debugging, durable replay, and human review pauses.

2. **Type-Safe Schema-Driven Agents (Pydantic AI, Atomic Agents).** Enforce strict validation on inputs, system prompts, dependency injection, and model tool calls using type systems (Pydantic, Zod). Guarantees that agent inputs and tool-call arguments strictly adhere to business domain types before execution.

3. **Role-Based Multi-Agent Teams (CrewAI, MetaGPT, PraisonAI).** Decompose complex objectives into specialized personas (e.g., Researcher, Coder, Critic) coordinated via hierarchical or peer-to-peer topologies. Agents exchange messages over shared blackboards or structured delegation protocols.

4. **Code-Writing and Lightweight Frameworks (smolagents, Agno).** Minimize abstraction layers by having the language model directly generate executable Python or TypeScript code blocks to express multi-step reasoning, bypassing complex JSON tool-calling wrappers.

5. **Enterprise Cloud-Native SDKs (OpenAI Agents SDK, Google ADK, Microsoft Agent Framework).** Provide vendor-backed, production-hardened SDKs designed for horizontal scaling, integrated enterprise identity, managed telemetry (OpenTelemetry), and direct coupling with cloud runtime infrastructure.

Key Selection Criteria

When choosing a framework, engineers evaluate four critical dimensions:

- **Persistence and Checkpointing**: Can the framework pause, store state in PostgreSQL/Redis, and resume after hardware failure?
- **Human-in-the-Loop**: Does it support native interrupt/resume primitives for sensitive tool authorizations?
- **Protocol Native**: Does it support MCP tool discovery and OpenTelemetry GenAI tracing out of the box?
- **Multi-Agent Topologies**: Can it coordinate hierarchical supervisors, peer swarms, and independent subagents without deadlocking?

KEY INSIGHT

Prefer explicit state machines over open-ended prompt loops. Frameworks like LangGraph and Pydantic AI that separate the execution state schema from prompt formatting consistently achieve higher reliability, lower token overhead, and simpler automated testing.

Pattern in Code: Approval gate with a checkpointed interrupt

LANGGRAPH 1.2 · PYTHON

```python
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import interrupt, Command

class State(TypedDict):
    request: str
    plan: str
    approved: bool

def plan(state: State) -> dict:
    return {"plan": f"refund order for: {state['request']}"}

def approve(state: State) -> dict:
    decision = interrupt({"plan": state["plan"]})   # pause for a human
    return {"approved": decision == "yes"}

def act(state: State) -> dict:
    return {"plan": state["plan"] + (" [executed]" if state["approved"] else " [rejected]")}

g = StateGraph(State)
g.add_node("plan", plan); g.add_node("approve", approve); g.add_node("act", act)
g.add_edge(START, "plan"); g.add_edge("plan", "approve")
g.add_edge("approve", "act"); g.add_edge("act", END)
app = g.compile(checkpointer=InMemorySaver())   # use PostgresSaver in production

cfg = {"configurable": {"thread_id": "ticket-42"}}
app.invoke({"request": "order 1009"}, cfg)        # runs until interrupt()
print(app.get_state(cfg).next)                     # ('approve',)
print(app.invoke(Command(resume="yes"), cfg)["plan"])
```

Ran successfully on 25 October 2026 with langgraph 1.2.17 on Python 3.13. Pin versions in your own project; these APIs change often.

Comprehensive Framework Catalog

START HERE

LangGraph (Python or TS), or Pydantic AI for single-agent services

Explicit state, checkpointing and interrupts are what keep agents debuggable in production; Pydantic AI gives the same discipline with less ceremony.

SWITCH WHEN You are all-in on one cloud (Google ADK, Strands, Microsoft Agent Framework) or a TypeScript web stack (Mastra, Vercel AI SDK).

TOOL	WHAT IT DOES · BEST FOR	LICENCE	LANG	STATUS
LangGraph FRAMEWORK	Cyclical state-machine orchestration with typed state schemas, persistent thread checkpointing, human-in-the-loop interrupts, and multi-agent graph composition. *Best for: Production agents requiring fault-tolerant state and human oversight.*	MIT	Py/TS	• Active
Pydantic AI FRAMEWORK	Model-agnostic, type-safe Python agent framework utilizing Pydantic for validation, dependency injection, streaming responses, and structured tool definitions. *Best for: Type-safe production backend microservices and API agents.*	MIT	Py	• Active
OpenAI Agents SDK SDK	Lightweight official Python/TS SDK featuring agent handoffs, guardrail wrappers, tool definitions, and automatic tracing. *Best for: Low-latency agents built primarily on OpenAI frontier models.*	MIT	Py/TS	• Active
Claude Agent SDK SDK	Anthropic SDK replicating the Claude Code agent loop with bash execution, file editing, and subagent delegation patterns. *Best for: Autonomous repository engineering and complex multi-file coding workflows.*	Mixed	Py/TS	• Active
Google ADK FRAMEWORK	Multi-language Agent Development Kit (Python, Java, Go, TypeScript) supporting Gemini function calling, code execution, and enterprise grounding. *Best for: Google Cloud and Vertex AI enterprise agent platforms.*	Apache-2.0	Multi	• Active
Microsoft Agent Framework FRAMEWORK	Successor to AutoGen and Semantic Kernel; reached 1.0 GA in April 2026 with graph workflows, checkpointing, MCP and A2A support. *Best for: Azure and .NET estates; new projects that would previously have chosen AutoGen or Semantic Kernel.*	MIT	Py/.NET	• Active
AWS Strands Agents SDK	Model-driven agent SDK where the model plans and calls tools in a loop; first-class Bedrock, MCP and multi-agent patterns (graph, swarm, agents-as-tools). *Best for: AWS-native agents and AgentCore deployments.*	Apache-2.0	Py/TS	• Active
CrewAI FRAMEWORK	Role-based multi-agent orchestration framework supporting hierarchical processes, autonomous agent delegation, task dependencies, and rich tool integrations. *Best for: Multi-role collaborative task automation (e.g., automated research teams).*	MIT	Py	• Active
LangChain FRAMEWORK	Component library for models, tools, retrievers and agents; now positioned as the high-level layer on top of LangGraph. *Best for: Teams that want batteries-included integrations.*	MIT	Py/TS	• Active
Deep Agents (LangChain) FRAMEWORK	High-level framework designed for long-horizon agent tasks involving multi-file reasoning, subagent management, and persistent planning memory. *Best for: Complex multi-hour engineering, analysis, and research workflows.*	MIT	Py/TS	• Active

TOOL	WHAT IT DOES · BEST FOR	LICENCE	LANG	STATUS
LlamaIndex Workflows FRAMEWORK	Event-driven, data-centric agent framework connecting multi-step agent reasoning with structured enterprise data pipelines and retrieval. *Best for: Complex RAG pipelines and knowledge-intensive agent systems.*	MIT	Py/TS	• Active
Mastra FRAMEWORK	Modern TypeScript-native agent framework featuring graph workflows, model routing, automated evaluations, and built-in local dev server. *Best for: Next.js, Node.js, and TypeScript full-stack AI applications.*	Apache-2.0	TS	• Active
Vercel AI SDK SDK	TypeScript library providing unified model abstractions, streaming UI generation, tool-calling pipelines, and direct frontend hooks. *Best for: Web applications, streaming chat interfaces, and interactive AI widgets.*	Apache-2.0	TS	• Active
smolagents LIBRARY	Hugging Face library where agents write executable Python code snippets to call tools and manipulate data, rather than generating JSON. *Best for: Lightweight data analysis and multi-modal tool use.*	Apache-2.0	Py	• Active
Agno FRAMEWORK	High-performance Python framework for building multimodal, multi-agent systems with integrated vector memory and session storage. *Best for: Fast conversational agents and enterprise copilots.*	MPL-2.0	Py	• Active
AutoGen / AG2 FRAMEWORK	AutoGen is in maintenance mode (bug and security fixes only) as Microsoft moves to Agent Framework; AG2 is the community fork that continues the conversational multi-agent API. *Best for: Existing AutoGen codebases and research prototypes; pick Agent Framework or AG2 for new work.*	MIT / Apache-2.0	Py	⁄ Legacy
Semantic Kernel SDK	Enterprise integration SDK from Microsoft bridging language models with conventional code plugins in C#, Python, and Java. *Best for: Existing C# and Java codebases; Microsoft directs new agent work to Agent Framework.*	MIT	Multi	○ Watch
DSPy FRAMEWORK	Programmatic framework that treats prompt engineering as an algorithmic optimization problem, compiling high-level pipelines into tuned prompts and weights. *Best for: Deterministic prompt optimization and structured extraction tasks.*	MIT	Py	• Active
BeeAI Framework FRAMEWORK	Open enterprise agent framework from IBM and Linux Foundation built for multi-agent workflows, governance, and model portability. *Best for: Enterprise-governed hybrid-cloud agent deployments.*	Apache-2.0	Py/TS	• Active
VoltAgent FRAMEWORK	Minimalist, high-throughput TypeScript agent runtime designed for high-concurrency serverless and edge environments. *Best for: Edge-deployed agents and serverless webhook automations.*	MIT	TS	• Active
Cloudflare Agents SDK SDK	Distributed, stateful agent runtime leveraging Cloudflare Workers, Durable Objects, and Vectorize for global edge execution. *Best for: Globally distributed edge agents requiring zero-cold-start state.*	MIT	TS	• Active
Inngest AgentKit SDK	Durable execution framework running agent networks on durable functions, preventing state loss during network timeouts or long waits. *Best for: Background agent jobs, webhooks, and asynchronous workflows.*	Apache-2.0	TS	• Active
Dapr Agents FRAMEWORK	Distributed runtime leveraging the Dapr virtual actor model for building resilient, stateful microservice agents across any cloud or Kubernetes. *Best for: Polyglot microservice architectures and high-scale enterprise systems.*	Apache-2.0	Py	• Active

TOOL	WHAT IT DOES · BEST FOR	LICENCE	LANG	STATUS
Spring AI FRAMEWORK	Java-native AI framework integrating modern LLM capabilities, vector stores, and prompt templates into the Spring enterprise ecosystem. *Best for: Enterprise Java backends and legacy enterprise infrastructure.*	Apache-2.0	Java	• Active
LangChain4j FRAMEWORK	Comprehensive Java library providing LLM integrations, document loaders, vector stores, and agent pipelines for Java and Kotlin developers. *Best for: Enterprise Java/Kotlin applications and microservice ecosystems.*	Apache-2.0	Java	• Active
Koog FRAMEWORK	Kotlin agent framework developed by JetBrains offering idiomatic coroutine support, type safety, and direct IDE integration. *Best for: Kotlin backends, Android agent workflows, and JetBrains plugins.*	Apache-2.0	Kotlin	• Active
Embabel FRAMEWORK	Goal-oriented agent framework for the JVM from Spring's creator; plans actions with typed domain models rather than free-form prompts. *Best for: Java and Kotlin teams wanting typed, testable agent planning.*	Apache-2.0	Kotlin	◦ Watch
Eino FRAMEWORK	Production-grade Go LLM application and agent framework developed by ByteDance for ultra-high concurrency and low-latency backends. *Best for: High-throughput Go microservices and high-scale cloud platforms.*	Apache-2.0	Go	• Active
Rig LIBRARY	Rust-native library for building fast, memory-safe LLM applications, embeddings pipelines, and agent workflows with minimal overhead. *Best for: High-performance systems programming and resource-constrained edge devices.*	MIT	Rust	• Active
Langroid FRAMEWORK	Multi-agent framework built around message-passing actor abstractions with first-class support for conversational multi-agent problem decomposition. *Best for: Complex multi-agent collaborative task solving in Python.*	MIT	Py	• Active
Atomic Agents FRAMEWORK	modular, schema-first Python agent library emphasizing separation of concerns, immutability, and deterministic validation. *Best for: Predictable, test-driven production pipelines with strict typing.*	MIT	Py	• Active
Griptape FRAMEWORK	Enterprise framework offering structured execution pipelines, memory offloading, and isolated data loaders for secure enterprise agents. *Best for: Secure cloud deployments with sensitive corporate data boundaries.*	Apache-2.0	Py	• Active
TaskWeaver FRAMEWORK	Code-first agent framework from Microsoft that interprets user requests into customized Python code executing stateful data analytics. *Best for: Advanced enterprise data analytics and domain-specific code generation.*	MIT	Py	◦ Watch
Rasa FRAMEWORK	Conversational assistant platform combining LLM reasoning with deterministic dialogue flows (CALM) for regulated customer conversations. *Best for: Contact-centre assistants that must follow auditable business logic.*	Apache-2.0 / Commercial	Py	• Active
CAMEL-AI FRAMEWORK	Multi-agent communicative framework pioneering role-playing cooperative and competitive agent simulations for complex research tasks. *Best for: Multi-agent research, societal simulation, and autonomous task exploration.*	Apache-2.0	Py	• Active

TOOL	WHAT IT DOES · BEST FOR	LICENCE	LANG	STATUS
MetaGPT FRAMEWORK	Multi-agent framework implementing Standardized Operating Procedures (SOPs) inspired by software development teams (Product Manager, Architect, Coder). *Best for: End-to-end automated software development from single prompts.*	MIT	Py	• Active
PraisonAI FRAMEWORK	Unified multi-agent platform combining CrewAI, AutoGen, and custom multi-agent structures under a simplified declarative YAML interface. *Best for: Rapid prototyping of multi-agent workflows and business automation.*	MIT	Py	• Active
AgentScope FRAMEWORK	Distributed multi-agent platform designed for high message-throughput, fault-tolerant communication, and enterprise multi-agent deployment. *Best for: High-concurrency enterprise multi-agent applications and simulation.*	Apache-2.0	Py	• Active
Swarms FRAMEWORK	Enterprise-oriented multi-agent orchestration library with many prebuilt topologies; fast-moving and API-unstable. *Best for: Experiments with large agent swarms.*	Apache-2.0	Py	◦ Watch
AutoGPT PLATFORM	The original autonomous-agent project, now a low-code platform for building and hosting continuous agents. *Best for: Hobbyist and prototype automation; historical reference.*	Mixed	Py/TS	◦ Watch
OpenAI Swarm LIBRARY	Educational multi-agent handoff library, superseded by the OpenAI Agents SDK. *Best for: Learning handoff patterns only.*	MIT	Py	⁄ Legacy

WATCH OUT

The Infinite Tool Oscillation Trap. When an agent has access to multiple tools without strict validation of intermediate progress, it frequently enters self-referential loops (e.g., repeatedly calling search with slight query variations). Always configure frameworks with explicit maximum step limits (max_iterations=15), timeout deadlines, and cyclical repetition detectors that abort execution if identical tool calls are observed consecutively.

Architectural Blueprint: Production Cyclical State Machines

Production-grade agent frameworks like LangGraph, Dapr Agents, and Pydantic AI replace unbounded prompt loops with formal, state-machine architectures comprising four critical structural primitives:

1. Typed State Schema. A centralized data structure (e.g., AgentState) tracking message histories, step counters, active scratchpads, and execution flags. Channel reducer functions define how new outputs merge into state without data loss or race conditions.

2. Computational Nodes. Isolated units of work (e.g., reason_node, tool_node) that ingest current state, perform a single deterministic action (such as invoking an LLM with bound tools or executing an MCP call), and return explicit state updates.

3. Conditional Edge Routing. Dynamic decision gates evaluated after each node execution. Conditional edges determine whether to proceed to tool execution (act), hand off to subagents, trigger human-in-the-loop interruption, or terminate the run (END).

4. Checkpoint Persistence. Snapshot storage engines (PostgreSQL, Redis, DynamoDB) that save state after every graph step. Checkpointing enables fault-tolerant recovery from network interruptions, full trajectory replay, and human approval pauses without memory loss.

PART III

03 Coding Agents and Development Environments

IDE agents, terminal harnesses, and cloud software engineers

Coding agents and agent development environments represent the most economically validated and rapidly deployed category in the agentic ecosystem. Rather than offering basic single-line code completions (classic tab-autocomplete), modern coding agents possess full repository awareness: reading directory trees, parsing ASTs, executing shell commands in sandboxes, running automated test suites, analyzing compiler and lint errors, and committing atomic multi-file diffs to version control.

The Three Operational Modalities

1. Terminal Harnesses (Claude Code, Gemini CLI, Aider, OpenCode). Operate directly inside the developer's shell environment. They parse local git status, edit files using specialized unified diff formats, invoke build commands, run test runners (pytest, cargo, vitest), and autonomously iterate until compiler errors are resolved.

2. Agentic IDEs (Cursor, Windsurf, Trae, Zed). Re-architect the code editor around the agent loop. Rather than treating chat as a side panel, these IDEs integrate deep semantic indexes, predictive edit prediction, terminal control, and instant diff acceptance natively into the editor buffer.

3. Autonomous Cloud Engineers (Devin, Google Jules, OpenHands). Operate asynchronously in isolated cloud microVMs. They receive a GitHub issue or feature request, create a development branch, plan the architectural change, write code across dozens of files, run CI tests, and submit a complete, validated pull request for human review.

Engineering Best Practices

- **Repo Map Precision**: High-performing coding agents generate dynamic abstract syntax tree (AST) maps that outline class hierarchies, function signatures, and dependency graphs without exhausting the context window.
- **Unified Diff Formats**: Instructing models to output surgical search-and-replace blocks or unified diffs prevents token bloat and avoids accidental file truncation common with full-file rewrites.
- **Deterministic Verification**: Always pair agent generation with deterministic feedback: linters, type checkers (tsc, mypy), and unit tests.

KEY INSIGHT

The highest predictor of coding agent success is not raw model parameter count, but the density and relevance of repository context provided to the model. Agents using AST-indexed symbol tables resolve complex cross-file dependencies with 40% higher accuracy than flat file search.

Comprehensive Coding Agents Catalog

START HERE

Claude Code or OpenAI Codex as the primary harness, with an AGENTS.md in every repo

Terminal-native harnesses run tests and fix their own failures; the instructions file makes results repeatable across tools.

SWITCH WHEN You need an IDE-first workflow (Cursor, Copilot) or fully open source and model-agnostic (OpenCode, Cline, Aider).

TOOL	WHAT IT DOES · BEST FOR	LICENCE	LANG	STATUS
Claude Code AGENT HARNESS	Anthropic's agentic coding harness for the terminal, IDEs, desktop and web: plans, edits files, runs commands and tests, delegates to subagents and extends through skills, plugins, hooks and MCP. *Best for: Terminal-first development, large refactors and autonomous test-fix loops.*	Proprietary	TS	• Active
OpenAI Codex (CLI, cloud, IDE) AGENT HARNESS	Command-line coding agent and delegated task execution harness tightly integrated with OpenAI frontier models. *Best for: Autonomous terminal scripting, system automation, and code generation.*	Apache-2.0 (CLI)	Rust	• Active
GitHub Copilot (agent mode, coding agent) PLATFORM	IDE and cloud agent modes integrating automated workspace context retrieval, multi-file edits, and automated terminal debugging. *Best for: Enterprise GitHub workflows, VS Code development, and pull request generation.*	Proprietary	n/a	• Active
Cursor IDE	Pioneering AI-first code editor fork of VS Code featuring shadow workspace indexing, Composer multi-file editing, and inline diff prediction. *Best for: Professional daily software engineering with tight AI editor integration.*	Proprietary	n/a	• Active
Google Antigravity PLATFORM	Google's agent-first development suite. Version 2.0 (June 2026) split into a conversational agent manager and a separate IDE, and its closed Antigravity CLI is replacing Gemini CLI. *Best for: Gemini-centric teams willing to absorb fast product churn.*	Proprietary	n/a	◦ Watch
Gemini CLI AGENT HARNESS	Open-source terminal agent for Gemini models with MCP and extensions; Google announced in June 2026 that the closed Antigravity CLI replaces it. *Best for: Existing users; plan a migration path before standardising on it.*	Apache-2.0	TS	◦ Watch
Google Jules PLATFORM	Asynchronous cloud coding agent running in managed environments, autonomously resolving GitHub issues and submitting verified pull requests. *Best for: Async issue resolution, dependency migrations, and routine maintenance.*	Proprietary	n/a	• Active
Amazon Kiro IDE	Spec-driven agentic development environment linking technical requirements directly to autonomous multi-file code generation. *Best for: Spec-first enterprise development and architecture enforcement.*	Proprietary	n/a	• Active
Amazon Q Developer PLATFORM	AWS-integrated coding assistant providing deep code reviews, automated security scanning, and legacy Java/SQL modernization. *Best for: AWS infrastructure as code, cloud modernization, and Java version upgrades.*	Proprietary	n/a	• Active

TOOL	WHAT IT DOES · BEST FOR	LICENCE	LANG	STATUS
Devin PLATFORM	Autonomous software engineering platform equipped with sandboxed browser, shell, code editor, and long-horizon planning engine. *Best for: End-to-end feature development, bug resolution, and complex migration tasks.*	Proprietary	n/a	• Active
Windsurf IDE	Agentic code editor developed by Codeium featuring Flows architecture that keeps the AI continuously synchronized with developer actions. *Best for: Teams already on Cognition (Devin), which acquired Windsurf in July 2025.*	Proprietary	n/a	• Active
OpenCode AGENT HARNESS	Open-source, terminal-based AI coding assistant supporting multiple model providers and local quantized models. *Best for: Developer terminal pair programming with model freedom.*	MIT	TS/Go	• Active
Goose AGENT HARNESS	Open-source autonomous developer agent from Block that automates complex multi-step coding and system configuration tasks. *Best for: Local, provider-agnostic automation; a founding project of the Agentic AI Foundation.*	Apache-2.0	Rust	• Active
Amp AGENT HARNESS	Next-generation repository-aware coding agent developed by Sourcegraph built on extensive code graph infrastructure. *Best for: Teams wanting a frontier-model coding agent; spun out of Sourcegraph as an independent company in December 2025.*	Proprietary	TS	• Active
Crush AGENT HARNESS	Terminal-native coding agent created by Charm emphasizing terminal ergonomics, TUI aesthetics, and fast local workflows. *Best for: Developer terminal environments and fast command-line iteration.*	FSL-1.1-MIT	Go	• Active
Qwen Code AGENT HARNESS	Terminal coding agent (forked from Gemini CLI) tuned for Qwen coder models, usable with local or hosted endpoints. *Best for: Private and air-gapped development on open models.*	Apache-2.0	TS	• Active
Aider AGENT HARNESS	Git-native AI pair programmer that writes surgical unified diffs directly into local git repositories and creates clean atomic commit messages. *Best for: Local git-based pair programming, rapid feature prototyping, and refactoring.*	Apache-2.0	Py	• Active
OpenHands PLATFORM	Leading open-source autonomous software engineering platform with Docker containerization, web UI, terminal, and browser capabilities. *Best for: Self-hosted autonomous engineering, custom agent research, and community benchmarks.*	MIT	Py	• Active
Cline IDE EXTENSION	Autonomous open-source coding agent extension for VS Code with strict human-approval gates for shell commands and file changes. *Best for: Secure, developer-supervised IDE agent coding with MCP tool support.*	Apache-2.0	TS	• Active
Roo Code IDE EXTENSION	configurable fork of Cline featuring custom agent personas (Architect, Coder, Ask), token tracking, and flexible system prompts. *Best for: Tailored developer workflows and specialized role-based coding tasks.*	Apache-2.0	TS	• Active
Kilo Code IDE EXTENSION	Open-source IDE agent offering multi-model support, fast local code index caching, and lightweight resource utilization. *Best for: Resource-constrained development machines and open-source tooling.*	Apache-2.0	TS	• Active

TOOL	WHAT IT DOES · BEST FOR	LICENCE	LANG	STATUS
Continue IDE EXTENSION	Open-source AI coding assistant for VS Code and JetBrains supporting custom slash commands, context providers, and self-hosted models. *Best for: Custom enterprise IDE extensions and local model integration.*	Apache-2.0	TS	• Active
Zed IDE	High-performance Rust-based code editor with native agent integration, Agent Client Protocol (ACP) support, and ultra-low latency. *Best for: High-speed developer workflows, collaborative coding, and Rust ecosystems.*	GPL-3.0	Rust	• Active
Warp TERMINAL	Modern Rust-based terminal incorporating agentic command generation, error explanation, and automated multi-step terminal workflows. *Best for: Modern command-line engineering and interactive terminal troubleshooting.*	Proprietary	Rust	• Active
JetBrains Junie / AI Assistant IDE AGENT	Native AI assistant for JetBrains IDEs (IntelliJ, PyCharm, WebStorm) leveraging deep Kotlin and Java AST project comprehension. *Best for: Enterprise Java, Kotlin, and Python development in JetBrains ecosystems.*	Proprietary	n/a	• Active
Augment Code PLATFORM	Enterprise-grade context engine and AI assistant capable of indexing multi-gigabyte codebases with real-time dependency tracking. *Best for: Large enterprise monorepos with hundreds of microservices.*	Proprietary	n/a	• Active
Factory (Droids) PLATFORM	Enterprise platform deploying specialized autonomous agents (Droids) for code migrations, automated testing, and code reviews. *Best for: Enterprise CI/CD automation, test generation, and compliance scanning.*	Proprietary	n/a	• Active
Trae IDE	AI-powered adaptive IDE from ByteDance offering built-in intelligent workspace indexing, chat, and automated multi-file code editing. *Best for: Full-stack web and mobile development with integrated AI tooling.*	Proprietary	n/a	• Active
Tabnine PLATFORM	Enterprise-focused AI code assistant providing air-gapped on-premises deployment, zero data retention, and strict IP indemnification. *Best for: Strictly regulated enterprise environments, defense, and banking.*	Proprietary	n/a	• Active
Replit Agent PLATFORM	Cloud-native natural-language-to-application builder that provisions environments, writes code, installs packages, and deploys live apps. *Best for: Rapid prototyping, MVP development, and non-technical software creation.*	Proprietary	n/a	• Active
Lovable / Bolt.new / v0 PLATFORM	Browser-based prompt-to-full-stack web application builders with live preview, WebContainer execution, and instant hosting. *Best for: Frontend UI prototyping, interactive landing pages, and full-stack web apps.*	Proprietary	n/a	• Active
SWE-agent FRAMEWORK	Autonomous issue-resolution harness developed by Princeton that interacts with repositories through an Agent-Computer Interface (ACI). *Best for: Benchmarking, autonomous issue fixing, and SWE-bench evaluation.*	MIT	Py	• Active
mini-SWE-agent FRAMEWORK	Minimal, modular reproduction of SWE-agent designed for reproducible research, lower token cost, and clean baseline comparisons. *Best for: Academic software engineering research and lightweight agent pipelines.*	MIT	Py	• Active

TOOL	WHAT IT DOES · BEST FOR	LICENCE	LANG	STATUS
Agentless FRAMEWORK	Non-agentic, deterministic pipeline for automated issue fixing that uses hierarchical localization followed by structured patch generation. *Best for: Cost-effective, deterministic bug fixing without cyclic loops.*	MIT	Py	○ Watch
Plandex AGENT HARNESS	Terminal-based AI engine for complex multi-file engineering tasks with branch-based version control and diff sandbox review. *Best for: Complex multi-stage refactoring and multi-file architecture changes.*	MIT	Go	○ Watch
Pythagora (GPT Pilot) PLATFORM	AI app-builder whose open-source core, GPT Pilot, pioneered step-by-step app generation with human checkpoints. *Best for: Prototype web apps with guided human review.*	FSL-1.1-MIT	Py	○ Watch
Sourcegraph Cody IDE EXTENSION	Code assistant now sold only to enterprise customers; free and Pro plans ended in July 2025 and the agent work moved to Amp. *Best for: Existing Sourcegraph Enterprise estates.*	Commercial	TS	⟋ Legacy
Mentat / Devika AGENT HARNESS	Early open-source coding-agent projects with little recent activity. *Best for: Historical reference only.*	Apache-2.0 / MIT	Py	⟋ Legacy

WATCH OUT

The Unconstrained Bash Trap. Granting coding agents unrestricted shell access without containerization or virtualized filesystem boundaries invites disastrous edge cases. A coding agent attempting to clean cache directories or parse complex filenames can easily issue destructive commands like `rm -rf /` or corrupt git histories. Always mandate microVM sandboxes (E2B, Docker) and require human-approval gates for irreversible operations.

04

PART IV

Foundation Models and Model Providers

the reasoning, perception, and cognitive engine

Foundation models are the reasoning engine of every agent. By late 2026 the frontier has moved to million-token context, native tool use and long-horizon agentic training, and the closed-versus-open gap has narrowed: the strongest open-weight models (Kimi K3, DeepSeek V4, Qwen3.8, GLM-5.3) now trail the closed frontier by months, not years. Model names change every few weeks, so this chapter states the lineup as of **25 October 2026** and teaches you to choose by role rather than by brand.

Four Roles in an Agent System

1. Planner and recovery model. The most capable model you can afford (Claude Opus 5.5, GPT-6 Astra or Sol, Gemini 4 Argon). Used for task decomposition, difficult tool sequences and recovering from failed steps. Few calls, high value per call.

2. Workhorse executor. A fast mid-tier model (Claude Sonnet 5.5, GPT-6 Luna, Gemini 3.8 Flash) that carries most tool calls in the loop. Optimise for tool-call reliability and latency, not leaderboard rank.

3. Open-weight model. Self-hosted when data cannot leave your network or unit cost dominates (Kimi K3, DeepSeek V4, Qwen3.8, GLM-5.3, Gemma 4, Mistral Large 3). Check the licence per model: several flagships use custom terms even when smaller tiers are MIT or Apache-2.0.

4. Small specialist models. Compact models that do one job inside the graph:

- **Embeddings** (Voyage, Cohere Embed, BGE, Qwen embeddings).
- **Rerankers** (Cohere Rerank, BGE-Reranker, Jina).
- **Vision-language** (Qwen3.8-VL, Gemma 4, Molmo) for screenshots and charts.
- **Guard models** (Llama Guard, ShieldGemma, Granite Guardian).

How to Choose

Run your own task-level eval (Part 13) on three candidates per role. Public leaderboards measure general ability; your agent fails on specific tool schemas, long contexts and domain vocabulary.

KEY INSIGHT

Tier your models. Route classification, extraction and argument checking to small, fast models and reserve the frontier model for planning and recovery. Most agent steps are routine, so this usually cuts inference spend substantially with no measurable loss in task success. Prove it on your own eval set before rolling it out.

Model Families and Providers Catalog

START HERE

One frontier model for planning plus one fast tier for execution, behind a gateway

Most steps are routine; a two-tier setup is cheaper and a gateway lets you swap models as releases land every few weeks.

SWITCH WHEN Data cannot leave your network: self-host Kimi K3, DeepSeek V4, Qwen3.8 or GLM-5.3 after checking each licence.

TOOL	WHAT IT DOES · BEST FOR	LICENCE	LANG	STATUS
Anthropic Claude PROVIDER	Claude Opus 5.5 (Sept 2026) is the default flagship, with Sonnet 5.5 and Haiku 5.5 below it. The Mythos tier sits above: Fable 5.1 is generally available with extra safeguards; Mythos 5.1 is restricted to vetted programmes. Opus and Sonnet offer 1M-token context. *Best for: Long-horizon coding agents, tool-heavy enterprise assistants, computer use.*	Proprietary	API	• Active
OpenAI GPT PROVIDER	GPT-6 family (Sept 2026): Astra at the top, Sol and Luna for cost tiers, GPT-6.1 Sol from DevDay. gpt-oss-120b and 20b are the open-weight tier (Apache-2.0). The Assistants API was sunset on 26 August 2026; build on the Responses API or Agents SDK. *Best for: General-purpose agents, realtime voice, structured outputs.*	Proprietary / Apache-2.0	API	• Active
Google Gemini PROVIDER	Gemini 4 Argon (generally available October 2026) leads; Gemini 3.8 Flash and 3.5 Flash-Lite cover latency and cost tiers, and 3.1 Pro remains available. Native multimodal input and 1M-token context. *Best for: Long-document and video understanding, Google Cloud agents.*	Proprietary	API	• Active
xAI Grok PROVIDER	Grok 4.7 (Sept 2026) is the most capable API model, with Grok 4.3 kept as a 1M-context option. *Best for: Real-time X data and reasoning workloads.*	Proprietary	API	• Active
Meta Muse and Llama PROVIDER	Meta replaced Llama with the closed Muse family in April 2026 (Muse Spark, Muse Code). Open weights continue through Muse Glimmer 30B (Apache-2.0). Llama 4 Scout and Maverick stay available under the Llama community licence but are not Meta's forward line. *Best for: Self-hosted Llama estates; evaluate Glimmer or other open families for new work.*	Proprietary / Open weights	API	◦ Watch
Mistral AI PROVIDER	Mistral Large 3 (675B MoE) and Small 4 ship as Apache-2.0 open weights, with Ministral 3 (3B to 14B), Devstral for coding and Voxtral for speech. Large 4 entered preview in October 2026. *Best for: EU data residency, permissively licensed self-hosting.*	Apache-2.0 / Commercial	API	• Active
DeepSeek MODEL	DeepSeek V4 Pro (1.6T total parameters, MoE) and V4.1 Flash (Sept to Oct 2026, 1M context, multimodal), all with MIT-licensed weights. *Best for: Low-cost reasoning and coding, fully self-hostable.*	MIT	Weights	• Active
Qwen (Alibaba) MODEL	Qwen3.8 (Aug 2026) open tiers, including a 27B dense vision-language model under Apache-2.0; the 2.4T flagship uses a custom licence and Qwen3.8-Max is hosted only. Qwen 4 is announced but not released. *Best for: Multilingual and coding agents on private infrastructure.*	Apache-2.0 / Custom	Weights	• Active
Moonshot AI (Kimi) MODEL	Kimi K3 (July 2026, 2.8T MoE) and K3.1 Pro (Oct 2026) lead open-weight agentic coding results; K2.7 Code targets software tasks. *Best for: Open-weight coding and research agents.*	Modified MIT	Weights	• Active

TOOL	WHAT IT DOES · BEST FOR	LICENCE	LANG	STATUS
Z.ai (GLM) MODEL	GLM-5.3 and GLM-5.3-Flash (Aug 2026); the Flash model is MIT-licensed, the full 5.3 uses a custom licence. *Best for: Agentic coding with permissive mid-size weights.*	MIT / Custom	Weights	• Active
MiniMax MODEL	MiniMax-M3 (June 2026) and M3.1 Flash (Oct 2026) with 1M-token context and low API pricing. *Best for: Long-context agents on a budget.*	Custom	Weights	• Active
Xiaomi MiMo MODEL	MiMo-V2.6-Pro (Sept 2026), a 1T-parameter MoE released with MIT weights. *Best for: Open-weight reasoning experiments.*	MIT	Weights	• Active
Google Gemma MODEL	Gemma 4 (April 2026): 31B dense and 26B MoE (about 4B active) plus edge variants, now Apache-2.0. *Best for: On-device and single-GPU agents, fine-tuning bases.*	Apache-2.0	Weights	• Active
NVIDIA Nemotron MODEL	Nemotron 3.5 Lightning 30B-A3B (Aug 2026) with open data and training recipes, tuned for tool calling and up to 1M context. *Best for: Efficient self-hosted agent workers on NVIDIA hardware.*	OpenMDW	Weights	• Active
IBM Granite MODEL	Granite 4.2 (3B, 8B, 30B; Aug 2026) plus Granite Guardian safety models, trained on documented enterprise data with IP indemnity. *Best for: Regulated enterprise backends and guard models.*	Apache-2.0	Weights	• Active
Ai2 OLMo MODEL	Olmo 3 and 3.1 publish weights, training data and code, the most open option for audit and research. *Best for: Research, auditability, reproducible fine-tuning.*	Apache-2.0	Weights	• Active
Microsoft MAI and Phi MODEL	First-party MAI models (MAI-Thinking-1, MAI-Code-1.1) on Microsoft Foundry, plus the MIT-licensed Phi-4 small models for local use. *Best for: Azure-native workloads and on-device helpers.*	Proprietary / MIT	Mixed	• Active
Amazon Nova MODEL	Nova 2 Lite for text and vision and Nova 2 Sonic for speech-to-speech, available through Bedrock. *Best for: Cost-sensitive AWS-native agents and voice.*	Proprietary	API	• Active
Cohere PROVIDER	Command A family for enterprise RAG and tool use, plus Embed and Rerank models widely used in retrieval stacks. *Best for: Private-cloud RAG and reranking.*	Proprietary	API	• Active
AI21 Labs PROVIDER	Jamba hybrid SSM-Transformer models with long context and low memory use. *Best for: Long-context document workloads.*	Jamba Open Model Licence	API	◦ Watch
Perplexity Sonar PROVIDER	Search-grounded model API that returns answers with citations. *Best for: Agents that need fresh, cited web answers.*	Proprietary	API	• Active
Amazon Bedrock PLATFORM	Managed access to Claude, Nova, Mistral, DeepSeek, Qwen and other families with guardrails, batch and provisioned throughput. *Best for: AWS estates needing one governed model API.*	Proprietary	API	• Active
Google Vertex AI PLATFORM	Managed Gemini plus partner models (Claude, Llama, Mistral) with Model Garden, tuning and evaluation. *Best for: Google Cloud estates.*	Proprietary	API	• Active
Microsoft Foundry PLATFORM	Renamed from Azure AI Foundry in late 2025; hosts OpenAI, MAI, Claude, DeepSeek and open models with agent service and evaluation. *Best for: Azure estates and Microsoft 365 integration.*	Proprietary	API	• Active
Hugging Face PLATFORM	Model and dataset hub with Inference Providers routing to partner backends, plus Inference Endpoints for dedicated deployments. *Best for: Discovering, evaluating and hosting open models.*	Mixed	API	• Active
NVIDIA NIM RUNTIME	Prepackaged, optimised inference microservices for popular models on NVIDIA GPUs. *Best for: Enterprise on-prem GPU deployments.*	Proprietary	Container	• Active
OpenRouter SERVICE	One API across hundreds of models from many providers with fallback and unified billing. *Best for: Model comparison, redundancy and prototyping.*	Proprietary	API	• Active

TOOL	WHAT IT DOES · BEST FOR	LICENCE	LANG	STATUS
Together AI / Fireworks AI SERVICE	High-throughput hosted inference and fine-tuning for open-weight models. *Best for: Production serving of open models without running GPUs.*	Proprietary	API	• Active
Groq / Cerebras / SambaNova SERVICE	Custom-silicon inference (LPU, wafer-scale, RDU) delivering very high tokens per second on open models. *Best for: Voice agents and fast inner loops.*	Proprietary	API	• Active
DeepInfra / Nebius Token Factory / Replicate SERVICE	Low-cost hosted open models and pay-per-call model execution. *Best for: Cheap batch generation and experiments.*	Proprietary	API	• Active

PART V

05 Model Serving, Inference, and Optimization

delivering low tail-latency and high-throughput execution

Model serving and inference runtimes provide the foundational computing infrastructure required to host foundation models with enterprise-grade reliability, predictable tail latency, and maximum hardware efficiency. For agentic systems, serving requirements diverge significantly from traditional single-turn chatbots: agents issue rapid sequences of repeated system prompts, complex tool schemas, and structured JSON requirements that place severe strain on memory and compute.

Key Inference Innovations

1. Prefix Caching and RadixAttention. Agent loops send identical system prompts and tool descriptions across 20+ turns. Engines like SGLang (RadixAttention) and vLLM retain KV cache tensors across requests in an LRU tree. Incoming agent turns match cached prefixes, reducing prefill time from seconds to milliseconds.

2. Grammar-Guided Constrained Decoding. Frameworks like Outlines, llguidance, and XGrammar compile JSON schemas and grammars into finite-state machines (FSMs) or pushdown automata. During sampling, invalid token logits are masked with zero latency penalty, guaranteeing 100% syntactically valid JSON tool calls.

3. Disaggregated Prefill and Decode (Splitwise, Mooncake). Separates compute-bound prefill (processing long agent prompt context) from memory-bandwidth-bound decode (generating tool call tokens), assigning them to dedicated GPU clusters.

Serving Architecture Checklist

- **PagedAttention**: Eliminates KV-cache memory fragmentation, enabling up to 4x larger concurrent batch sizes.
- **Continuous Batching**: Iteration-level scheduling that inserts new agent requests into active generation batches without waiting for long sequences to finish.
- **Speculative Decoding**: Using small draft models (or Medusa/EAGLE heads) to generate multiple candidate tokens verified in parallel by the target model.

KEY INSIGHT

Enforce grammar-constrained decoding (via Outlines, XGrammar, or vLLM guided decoding) for all tool-calling agent nodes. Eliminating JSON parse failures at the engine level removes the need for expensive retry loops and prevents agent crashes.